

ONLINE EXAMINATION SYSTEM

Product Requirements Document (PRD)

Subject: Software Engineering

Document Version: 1.0

Project Type: Academic / Web Application

Development Approach: Agile, Sprint-based

A clean, simple and production-structured online examination platform for students, faculty and administrators.

This document is the source of truth for the project. Implementation prompts should follow this PRD and should not silently change product scope or architecture.

1. Document Purpose

This PRD defines what the Online Examination System should do, who will use it, how users will move through the product, what screens and states are required, and what technical constraints should guide implementation. It is intentionally written so the same document can support development, testing, documentation and Software Engineering viva/project evaluation.

Primary principle: simple UX, clear information hierarchy, predictable behavior, and maintainable code.

2. Product Vision

Build a lightweight web-based examination platform where students can discover and attempt exams without confusion, while faculty can create and evaluate exams with minimal administrative work. The system should feel like a deliberately designed academic product rather than a collection of CRUD screens.

3. Goals

- Allow authenticated students to view eligible exams and attempt them online.
- Allow faculty/examiners to create exams, manage questions and publish exams.
- Automatically evaluate objective questions and generate results.
- Give students a clear result and answer-review experience.
- Give administrators controlled visibility over users, exams and system data.
- Maintain secure role-based access and server-side validation.
- Keep the interface responsive, accessible and easy to learn.
- Provide enough engineering structure for testing, documentation and future extension.

4. Non-Goals / Scope Boundaries

- No live video proctoring in MVP.
- No AI-based cheating detection in MVP.
- No complex subjective-answer grading in MVP.
- No payment/subscription system.
- No public anonymous exam attempts.
- No unnecessary social/community features.
- No dependence on paid proprietary APIs for core functionality.

5. Target Users & Roles

Role	Purpose	Key Capabilities
Student	Takes examinations	View exams, read instructions, attempt questions, save progress, submit, view re
Faculty / Examiner	Creates and manages exams	Create exams, manage question bank, configure exam, publish/unpublish, view r
Admin	System administration	Manage users, subjects, exams, access, system-level visibility

6. Core User Journeys

Student: Login → Dashboard → Available Exam → Exam Instructions → Start Exam → Answer Questions → Review → Submit → Result.

Faculty: Login → Faculty Dashboard → Create Exam → Add Questions → Configure Settings → Preview → Publish → Monitor Results.

Admin: Login → Admin Dashboard → Users / Subjects / Exams → Manage → Audit basic system state.

7. Functional Requirements

ID	Requirement	Description
FR-01	Authentication	System shall support secure login/logout and identify the user's role.
FR-02	Authorization	Users shall only access features permitted for their role.
FR-03	Student Dashboard	Students shall see relevant upcoming, available and completed exams.
FR-04	Exam Discovery	Students shall see exam title, subject, duration, total marks, question count and availability.
FR-05	Exam Instructions	Before starting, students shall see clear instructions and exam metadata.
FR-06	Exam Attempt	Students shall answer objective questions and navigate between questions.
FR-07	Question Palette	Students shall see answered/unanswered/marked-for-review states.
FR-08	Auto-save	Answers shall be persisted during an active attempt to reduce accidental data loss.
FR-09	Timer	Active exams shall display remaining time and enforce server-side expiry.
FR-10	Submission	Students shall explicitly submit an exam; expired exams shall be auto-submitted according to
FR-11	Evaluation	Objective questions shall be evaluated automatically.
FR-12	Results	Students shall see score, percentage, time information and question-level outcome where allo
FR-13	Question Bank	Faculty shall create, edit, delete and organize objective questions.
FR-14	Exam Builder	Faculty shall configure title, subject, duration, questions, marks and availability.
FR-15	Publishing	Faculty shall preview and publish an exam; published exams shall follow controlled editing rule
FR-16	Faculty Results	Faculty shall view results for exams they own/manage.
FR-17	User Management	Admin shall manage users and roles within the defined permissions.
FR-18	Subject Management	Admin/faculty permissions shall support subject organization as defined by implementation.
FR-19	Validation	All critical input shall be validated client-side for UX and server-side for security.
FR-20	Error Handling	The application shall provide useful loading, empty, error and success states.

8. Non-Functional Requirements

- Usability: A first-time student should understand how to start an available exam without training.
- Performance: Normal dashboard and exam interactions should feel responsive; avoid unnecessary network requests.
- Security: Passwords must be hashed; authorization must be enforced server-side; users must not trust client-provided role/score data.
- Reliability: Active answers should be persisted safely and exam expiry must be handled consistently.
- Maintainability: Use modular components/services, clear naming, validation and separation of concerns.
- Accessibility: Keyboard-accessible controls, visible focus, readable contrast, labels and semantic structure.
- Responsive design: Student exam flow must work on desktop and mobile; dense admin tables may adapt into cards on small screens.
- Observability: Development environment should have structured error logging without exposing secrets.

9. MVP Feature Set

Area	MVP
Auth	Login, logout, role-based access
Student	Dashboard, exam list, instructions, attempt, timer, navigation, auto-save, submit, result
Faculty	Dashboard, question bank, create/configure exam, preview, publish, results
Admin	Basic user/role management and system overview
Questions	MCQ / single-correct objective questions
Results	Automatic scoring, percentage, basic breakdown
UX	Responsive layout, loading/empty/error/success states

10. Screen Inventory

Area	Screen	Purpose
Public/Auth	Login	Role-aware sign-in; clear errors; loading; session handling
Student	Dashboard	Exam summary, upcoming/available/completed sections
Student	Exam Details	Metadata and eligibility; Start action
Student	Instructions	Rules, duration, marks, attempt policy; confirmation
Student	Exam Workspace	Question, options, palette, timer, navigation, mark-for-review
Student	Submit Confirmation	Answered/unanswered summary and final confirmation
Student	Result	Score, percentage, time, breakdown
Student	Answer Review	Question-wise outcome where exam policy allows
Faculty	Dashboard	Exam status, quick actions, result overview
Faculty	Question Bank	Search/filter/create/edit/delete questions
Faculty	Exam Builder	Details, questions, settings

Area	Screen	Purpose
Faculty	Exam Preview	Student-like preview before publishing
Faculty	Exam Results	Attempts, scores and basic statistics
Admin	Admin Dashboard	Users, exams, subjects, system overview
Admin	User Management	Create/update/deactivate users and assign roles

11. UX / UI Design System

Visual direction: minimal academic SaaS. Use a neutral base, one primary accent, restrained status colors, generous spacing and strong typography. Avoid decorative gradients, excessive shadows, oversized hero sections and unnecessary animations.

- Layout: consistent max-width containers, predictable sidebar/topbar patterns by role.
- Typography: clear hierarchy with 1 display scale, 2–3 heading levels, readable body text.
- Components: buttons, inputs, selects, tabs, cards, badges, modal/dialog, toast, table, pagination, skeleton, empty state.
- Exam workspace: content-first layout; timer always visible; navigation controls easy to reach.
- Feedback: every async action should communicate loading/success/failure where relevant.
- Destructive actions: delete, unpublish, deactivate and final submission require appropriate confirmation.
- Consistency: same component should behave and look the same across roles.

12. Critical UI States

- Loading: skeletons or purposeful loading indicators instead of blank screens.
- Empty: explain why there is no data and provide a relevant next action.
- Error: human-readable message plus retry/back navigation when useful.
- Unauthorized: explain that the user does not have permission; never reveal protected data.
- Offline/interrupted request during exam: preserve local state where safe and reconcile with server state.
- Expired exam: lock further answering and show submission/result state according to policy.
- Unsaved faculty changes: warn before leaving when there are meaningful unsaved changes.

13. Exam Rules & Business Logic

- An exam has a configured duration and availability window.
- A student can start only if the exam is published and the student is eligible.
- The server records the official attempt start time.
- Remaining time should be derived from server-side timestamps, not only a client countdown.
- Answers should be saved incrementally during an active attempt.
- Final submission is idempotent: repeated submission requests must not create multiple results.
- Once submitted or expired, an attempt becomes immutable except for controlled administrative operations.
- Score must be calculated from authoritative question/answer data on the server.
- If question editing is restricted after publication, result integrity must be preserved.

14. Suggested Data Model

Entity	Important Fields
User	id, name, email, passwordHash, role, status, createdAt, updatedAt
Subject	id, name, code, description, status
Question	id, subjectId, text, type, marks, explanation, status, createdAt
QuestionOption	id, questionId, text, isCorrect
Exam	id, title, subjectId, description, duration, totalMarks, status, startAt, endAt, createdBy
ExamQuestion	id, examId, questionId, order, marks
Attempt	id, examId, studentId, startedAt, submittedAt, status, score, percentage
AttemptAnswer	id, attemptId, questionId, selectedOptionId, savedAt, markedForReview

15. API / Backend Boundary

Use REST-style endpoints or an equally clear API contract. Keep authentication, authorization, validation and business logic on the server. Do not allow the client to submit a trusted score, role or attempt ownership.

Area	Representative API Contract
Auth	POST /auth/login, POST /auth/logout, GET /auth/me
Student	GET /exams, GET /exams/:id, POST /attempts, GET /attempts/:id, PUT/PATCH answer, POST /attempts/:id/su
Faculty	CRUD /questions, CRUD /exams, publish/unpublish, GET /exams/:id/results
Admin	User management, subject management, system overview

16. Recommended Open-Source Stack

Layer	Recommendation	Reason
Frontend	React + TypeScript	Component-based, maintainable, widely supported
Styling/UI	Tailwind CSS + shadcn/ui	Fast consistent UI without locking into a proprietary design system
Backend	Node.js + Express	Simple API architecture suitable for an academic project
Database	PostgreSQL	Reliable relational model for exams, attempts and results
ORM	Prisma	Typed database access and migrations
Validation	Zod or equivalent	Shared/clear schema validation
Testing	Vitest + Playwright	Unit/component + end-to-end coverage
Tooling	Git + GitHub	Version control and collaboration
Containerization	Docker	Reproducible local setup

17. Architecture

Recommended architecture: browser client → REST API → service/business layer → Prisma/ORM → PostgreSQL. Keep UI components separate from API calls, API routes separate from business rules, and

database access behind a clear data layer. Authentication middleware should protect private routes; authorization checks should occur at the server boundary and in sensitive service operations.

Suggested repository structure:

```
online-examination-system/
├── apps/
│   ├── web/
│   └── api/
├── packages/
├── shared/
├── docs/
├── tests/
├── docker/
├── .env.example
├── README.md
└── package.json
```

18. Sprint Plan

Sprint	Focus	Deliverables	Exit Criteria
Sprint 0	Requirements & Planning	PRD, scope, user stories, use cases, acceptance criteria	Approved requirements
Sprint 1	UX & Architecture	Wireframes, design system, DB/ERD, architecture, API contract	Design & architecture baseline
Sprint 2	Foundation & Auth	Repo setup, database, migrations, auth, roles, base workflow	Working role-based login
Sprint 3	Exam Core	Question bank, exam builder, student attempt flow, timer, auto save	End-to-end exam attempt
Sprint 4	Evaluation & Results	Scoring, submission, results, faculty result view	Complete assessment loop
Sprint 5	Admin, Security & Polish	Admin tools, validation, security hardening, UX state, responsiveness	Secure, responsive MVP
Sprint 6	Testing & Release	Unit/E2E tests, bug fixes, documentation, deployment, release candidate	Release candidate

19. Testing Strategy

- Unit tests: scoring rules, validation, permission checks, time/expiry logic.
- Integration tests: authentication, exam creation, attempt creation, answer persistence, submission.
- End-to-end tests: student completes an exam; faculty creates/publishes exam; result is generated.
- Negative tests: unauthorized access, invalid IDs, expired exams, duplicate submission, tampered score/role, malformed payloads.
- UI tests: keyboard navigation, responsive layouts, error and empty states for critical screens.

20. Acceptance Criteria — Core Flow

- Student can log in with valid credentials and is redirected to the correct dashboard.
- Student cannot access faculty/admin routes.
- Faculty can create a draft exam and add valid MCQ questions.
- Faculty can preview and publish an exam.
- Published exam appears to an eligible student.
- Student can start the exam and the server records the start time.
- Student can answer, navigate, mark for review and see saved state.
- Refreshing the exam page does not silently erase saved answers.
- When time expires, the system prevents further answering and finalizes the attempt according to the configured policy.
- Manual submission creates exactly one final result.
- Score is calculated server-side from the authoritative answer key.
- Student can view the resulting score and percentage.
- Faculty can view results for their exam.
- Unauthorized users cannot retrieve another student's private attempt/result.
- Critical failures show a recoverable, human-readable error.

21. Security Requirements

- Never store plaintext passwords; use a modern password hashing algorithm.
- Protect private API routes with authentication middleware.
- Enforce role/ownership authorization server-side.
- Validate and sanitize incoming data.
- Do not expose correct answers before an attempt is finalized unless policy explicitly allows it.
- Do not trust client-provided score, role, exam ownership or remaining time.
- Use secure cookie/token practices appropriate to the selected auth architecture.
- Keep secrets in environment variables and commit only .env.example.
- Use parameterized/ORM queries and avoid raw SQL unless safely parameterized.
- Log security-relevant errors without logging passwords, tokens or sensitive answer data.

22. Academic Software Engineering Deliverables

- Problem Statement
- Project Objectives and Scope
- SRS / Requirements Specification
- Use Case Diagram
- Activity Diagrams for major flows

- Sequence Diagrams for login, exam attempt and submission
- ER Diagram / Database Schema
- System Architecture Diagram
- UI Wireframes / Screens
- Test Plan and Test Cases
- Risk Register
- Sprint / Iteration Records
- Final Project Report
- Installation / Deployment Guide
- User Guide
- Demo Script

23. Definition of Done

- Feature works against the defined acceptance criteria.
- Happy path and important failure paths are tested.
- Authorization is verified server-side.
- No known critical console/API errors remain.
- Responsive behavior is checked on desktop and mobile widths.
- Loading, empty and error states exist where applicable.
- Code follows repository conventions and has no unnecessary duplication.
- Relevant documentation is updated.
- Feature is committed with a clear Git message and can be demonstrated.

24. Risks & Mitigations

Risk	Mitigation
Scope creep	Treat this PRD as baseline; defer non-MVP ideas to a backlog.
Timer manipulation	Use server timestamps and server-side expiry.
Data loss during exam	Incremental persistence plus safe local UI state where appropriate.
Unauthorized result access	Server-side ownership and role checks.
Question changes after publication	Restrict edits or version/snapshot exam content.
Over-complex UI	Use a small design system and consistent components.
AI-generated inconsistent code	Implement in small prompts, review every change, run tests after each milestone.

25. Development Rules for AI-Assisted / Vibe-Coded Implementation

- AI is an implementation assistant, not the source of truth. This PRD remains authoritative.
- Implement one bounded task at a time; do not generate the entire application in one prompt.
- Before changing architecture, database schema or core UX, explicitly update the PRD/decision record.
- Do not invent libraries when an existing project dependency already solves the problem.
- Do not duplicate components or create multiple competing patterns for the same UI behavior.
- After each sprint, run tests, inspect the diff, clean unused code and update documentation.

- Prefer boring, readable code over clever abstractions.
- Every feature must have a clear user value and acceptance criteria.

26. Implementation Sequence

The project should be built in this order:

- 1. Lock requirements and architecture.
- 2. Create repository and development conventions.
- 3. Build database schema and migrations.
- 4. Build authentication and role authorization.
- 5. Build shared UI shell and design system.
- 6. Build faculty question bank.
- 7. Build faculty exam creation/publishing.
- 8. Build student exam discovery and instructions.
- 9. Build exam workspace, timer and autosave.
- 10. Build submission and scoring.
- 11. Build results and faculty result views.
- 12. Build admin essentials.
- 13. Add tests/security/polish.
- 14. Deploy and produce academic documentation.

27. Backlog — Future Enhancements

- Randomized question order / option order.
- Question pools and randomized exam generation.
- Multiple question types.
- CSV/Excel question import/export.
- Detailed analytics and performance trends.
- Email notifications.
- Certificate generation.
- Advanced audit logs.
- Accessibility enhancements beyond MVP.
- Optional proctoring integrations, only if project scope and privacy requirements permit.

28. Final Product Principle

Every screen should answer one obvious question for the user: “What can I do here, and what should I do next?” The product should remain small, predictable and useful. Complexity should live in the architecture and business rules—not in the user's interface.

END OF PRD — VERSION 1.0